Lossless KV Cache Compression for LLM Serving: Exact-Length Records in the Cache Tiers and the GPU Pool

[TBD: authors; at least one non-academic author required by the CFP]

***Abstract*—LLM serving engines spill the attention key/value (KV) cache from GPU memory to host DRAM and NVMe so that agent and multi-turn requests skip recomputing shared prefixes. Whether a returning session still finds its prefix, and how many long requests the GPU runs at once, then set time-to-first-token (TTFT) and goodput. Lossy KV compression enlarges both but changes what the model attends over. ExactKV integrates a lossless KV code into both places without changing vLLM: in LMCache's cache server (tier mode), GPU kernels encode each chunk into an exact-length record that the existing DRAM and NVMe tiers store, evict and prefetch unchanged, and decode it back into vLLM's paged blocks with no host synchronization; as a vLLM attention backend (pool mode), records packed into vLLM's pages are read in place by a fused decode-attention kernel. The code, a two-level 3-bit code for the sign and exponent, stores BF16 KV at 0.71 of its bytes on nine models. On the AgentX benchmark (Qwen3-30B-A3B on two H800s; GLM-4-9B reproduces the cliff and the pool result) a compressed tier acts as a raw tier of 1.4× its bytes, with bit-identical outputs: on 880 GiB of DRAM plus NVMe it matches a raw deployment of 1231. On DRAM alone, LRU over tier records caches more prompt tokens than offline-optimal replacement over raw bytes, and the codec serves 3.5–4.3× raw's goodput between the two cliffs. With NVMe, LMCache's write-through LRU needs about 1.8× the live KV; at 1.4–1.6× the codec serves 2.8–3.3× raw's goodput over a 30-minute window, a gain an exclusive tier policy would mostly recover in simulation. On FP8 KV past its DRAM cliff it raises goodput 1.8–2.9×. A compressed GPU pool performs as a raw pool of its 1.31–1.34× token count; for BF16 KV its decode steps are no longer than FlashAttention-3's.**

***Index Terms*—LLM serving, KV cache, lossless compression, tiered storage, GPU**

I. Introduction

Prefix reuse saves much of the work in production LLM serving: agent loops re-send a growing transcript every turn, and chat and retrieval-augmented (RAG) requests share history and documents [19], [21]. KV caches are large: Qwen3-30B-A3B stores 96 KiB per token, so a GPU that reserves 32 GiB for KV fits five 64K-token agent sessions. Engines such as vLLM [9] and SGLang [10] therefore spill KV blocks to host DRAM and NVMe through external tiers (LMCache [1], Mooncake [2], SGLang HiCache [50], CachedAttention [3], Strata [11]) and load them back on a prefix hit.

Four observations. Fig. 1 measures raw KV on our testbed. *Capacity* (a): agent sessions return in turn after long gaps, so LRU tiers that cannot hold the sessions' KV evict each prefix shortly before it is reused. With 64 AgentX sessions [20] (641 GiB of live KV on average) and 400 GiB of DRAM, raw LMCache on 2 TiB of NVMe serves 0.94 of prompt tokens from a cache and 15.1 requests per minute within the target (engine TTFT ≤2 s, ITL ≤40 ms); on 640 GiB it caches 0.55 and serves 4.6 (1040 in total). *Inference* (b): a 96K-token prompt holds 4.5 GiB of KV on each of two GPUs, so once the pool binds, requests wait in vLLM with their prefixes cached; only a pool that attention reads in place compressed adds requests in flight. *Transfer* (c): a hit from DRAM or NVMe costs 3–19 μs per prompt token against 35–93 to recompute, so what matters is that the hit exists [16]. *Data* (d): seven values cover 94% of a plane's BF16 high bytes, so a lossless code could store KV at about 0.7 of its bytes. We call a compressed level *capacity-equivalent* if, holding B bytes at stored ratio r, it gives the results of a raw level of B/r bytes; this paper asks whether storing, loading and attending over records keep that equivalence, and what it buys.

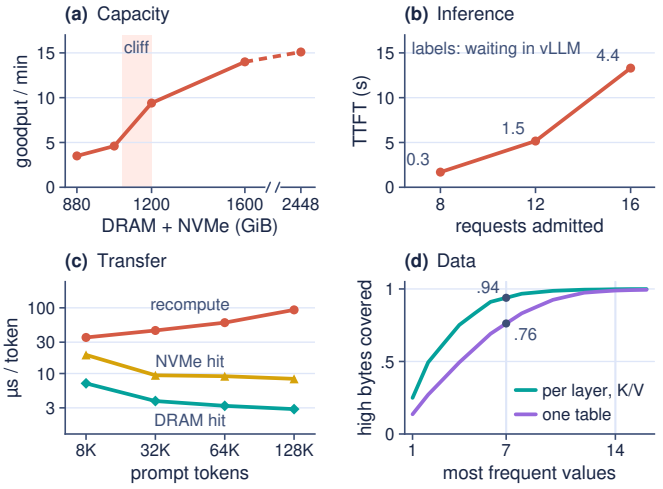


Fig. 1. Four observations on raw KV (Qwen3-30B-A3B, TP 2 on two H800s). (a) Capacity: AgentX goodput of raw LMCache (requests per minute with engine TTFT ≤2 s and ITL ≤40 ms; 64 sessions, 641 GiB of live KV, 400 GiB DRAM) by DRAM plus NVMe, as in Fig. 7. (b) GPU pool: with every prefix cached (2 TiB NVMe, 54 GiB of KV per GPU), mean engine TTFT by requests admitted at once; labels: requests waiting in vLLM. (c) Transfer: time per prompt token of a full hit loaded from DRAM or NVMe, and of recomputing the prompt. (d) Data: share of BF16 high bytes covered by the most frequent values, with one table per layer and K/V or one for the tensor (labels: at seven values); the low byte carries 7.97 of 8 bits.

Lossless vs lossy. Lossy KV compression (KIVI [22], FP8 KV, CacheGen [4], KVTC [57]) stores fewer bytes but changes the values, so accuracy is requalified per model and task, and with tiers a request's values depend on the tier that served

it. ExactKV stores the given KV format exactly: in tier mode a hit gives the outputs of a raw tier with the same hit, and pool mode attends with another kernel, qualified once like any backend. The two compose: ExactKV stores FP8 KV at 0.85 of its bytes.

Challenges. (R1) *Ratio:* no order-0 code goes much below 0.67 of BF16 or 0.80 of FP8, so the format has little room for overhead. (R2) *Paged restore:* the engine's cache is scattered blocks, and a separate decode buffer costs a copy larger than the decode. (R3) *Off the critical path:* a record's length depends on its data and must not be learned on the host during a step or load; GPU compressors (nvCOMP [30]) leave R2 and R3 to the caller. (R4) *Inside an existing tier:* the tier must keep each record at its length without the codec taking over its allocation, eviction or prefetch. (R5) *Attention over compressed KV* at FlashAttention's speed [45].

Contributions. ExactKV runs with unmodified vLLM 0.30, inside the cache server of LMCache 0.5.6 [1] and as an out-of-tree vLLM attention backend.

- **Lossless records inside an existing tier** (§V): encodes run on a worker stream, each record gets one L_1 object of its exact length, and loads decode in place into vLLM's paged blocks with no host synchronization; LMCache's allocation, eviction, write-through and prefetch are unchanged.
- **A compressed GPU pool** (§VI): records laid out in the decode kernel's thread order are packed into vLLM's pages and attended over in place; decode steps never synchronize with the host, so they replay as CUDA graphs.
- **A paged-native exponent code** (§IV), in the family of ZipServ's [35]: a second escape level stores 1.8% fewer bytes than its one-level code, and tiles decode independently straight into scattered blocks.
- **An evaluation in two steps** (§VIII), tier capacity then GPU capacity, on AgentX against LMCache with its best settings (LMCache 0.5.6 offers no exclusive or scan-resistant tier policy), with a policy simulation of what the results depend on, and an operator's sizing rule (§X).

Results. Compressed tiers are capacity-equivalent at their ratio r: tier mode on 880 GiB of DRAM plus NVMe matches a raw control of 1231, bin by bin. On DRAM alone the gain survives any replacement policy: LRU over tier records beats Belady's over raw bytes between the two cliffs. Under LMCache's write-through LRU with NVMe, raw tiers need about 1.8× the live KV; on 1040 GiB tier mode keeps 0.91 of prompt tokens cached against 0.55 and serves 2.8–2.9× the goodput (3.0–3.3 on 880). That multiplier belongs to LMCache's inclusive policy at its cliff (§X). A compressed GPU pool is capacity-equivalent at its token count. Code: https://github.com/GentleCold/LMCache/tree/lossless-kv.

II. Background and Motivation

A. External KV tiers in vLLM

vLLM stores KV in a paged pool of 128-token blocks shared by all requests. A KV connector (`KVConnectorBase_V1`) has two halves. On admission the scheduler half reports how many prompt tokens beyond the GPU prefix cache are available externally; vLLM allocates blocks and, with asynchronous loading, parks the request until the worker half reports the load done. After each step the worker half may save new blocks. Placement is the connector's; the engine only sees block IDs.

LMCache's cache server. In its multiprocess mode, LMCache runs one cache server per node next to the engine. Each vLLM worker registers its paged KV cache by CUDA IPC handles, so the server moves blocks between the engine's GPU memory and its own tiers with its own kernels and streams: L_1 is a pinned DRAM pool, L_2 one or more adapters (here a native file adapter with O_DIRECT on NVMe). Stores land in L_1 and are written through to L_2 asynchronously; a lookup prefetches L_2-only chunks into L_1 and the load copies them from L_1 into the engine's blocks. A once-a-second loop evicts least-recently-used L_1 objects when occupancy passes a watermark. Every object is a fixed layout of the raw chunk.

B. Where the compressible bits are

A BF16 scalar's high byte holds the sign and the top seven exponent bits, its low byte the last exponent bit and seven mantissa bits; an FP8 E4M3 byte is a sign, four exponent bits and three mantissa bits. We dumped the KV of 4096 tokens of agent text from Qwen3-30B-A3B and GLM-4-9B (201 M and 168 M scalars), fitted tables on the first 1024 tokens and measured the rest (Fig. 2). The low byte is noise: it carries 7.97 of 8 bits (7.82 given the high byte) and FP8's three mantissa bits 2.97, so the redundancy is in the sign and exponent. A 9-bit cut that also codes the last exponent bit could save at most the 0.18 bits that bit shares with the high byte; the byte cut keeps every stream byte-aligned and every symbol a byte-permute selector (§IV). The high byte carries 3.57 bits over the whole tensor but 2.84 within one layer and one of K/V (a *plane*), because each plane concentrates on a few values: the seven most frequent cover 93.9% of scalars, the next seven 5.9%, and 0.19% fall outside fourteen. The order-0 bound is therefore 0.721 of raw with one global table and 0.676 with one table per plane (Fig. 2b). Finer tables do not help BF16: per head it stays 0.676, and per-channel tables fitted on the first tokens score 0.679 on the rest; byte-shuffling zstd reaches 0.687 on the CPU. FP8 channels of one head sit at different exponents, so a per-head table over exponents aligned by a per-channel offset gives 0.795, as per-channel tables do. Conditioning on the previous token would reach 0.654 and 0.758, but decodes serially. Of the removable 0.32 of BF16 (0.21 of FP8), ExactKV's tier records remove 0.29 (0.15): its fixed-length code costs 0.024 (0.034) over the bound, of which a per-head Huffman code would recover 0.020 (0.024) at the price of variable-length decoding, and its tile index 0.14 bits per scalar, 0.009 of BF16 but 0.026 of FP8 (§IV).

C. Why not an existing codec

nvCOMP's codecs treat KV as bytes, need contiguous input and report lengths through the host. Weight codecs [31], [32], [35] compress offline; ZipServ's code is the nearest in form: a

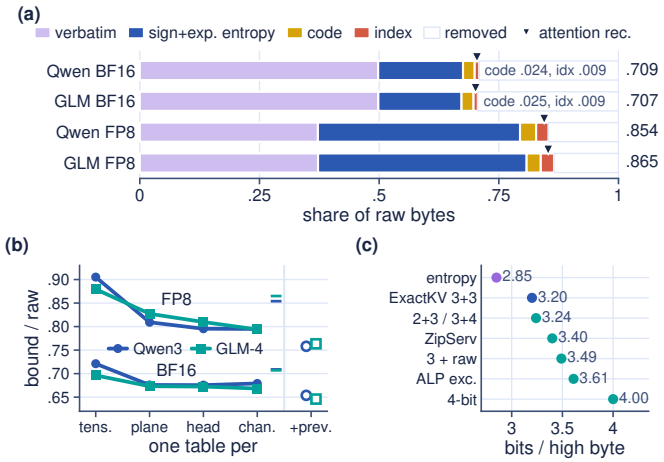


Fig. 2. Where the compressible bits are (Qwen3-30B-A3B and GLM-4-9B KV, tables fitted on the first 1024 tokens, measured on 3072). (a) Raw bytes split into the verbatim bits (BF16 low byte, FP8 mantissa), the entropy of sign and exponent per head, what ExactKV's fixed-length code and its tile index add, and what the tier record removes (label: stored ratio; triangle: FragPack record). (b) Order-0 bound by table granularity (FP8 per head after per-channel exponent offsets; per-channel tables scored on held-out tokens); hollow: also conditioned on the previous token, which a parallel decoder cannot use; ticks: stored tier ratio. (c) Bits per BF16 high byte by escape scheme (§IV).

3-bit code per BF16 exponent over a window of seven values, with full-precision fallback, decoded into tensor-core operands inside GEMM. Applied per plane to our KV, its high part costs 3.40 bits against 3.20 for ours, so with the same tile index it would store 0.722 of raw against ExactKV's 0.709 (Table II). Unlike weights, KV is encoded online, restored into paged blocks and decoded inside attention. SplitZip [5] codes BF16 exponents with 4-bit symbols for transfer, with no paged destination or tier. A codec in front of a tier adds no capacity by itself: tier stores size objects for raw chunks until the store keeps a length per record (§V).

III. System Overview

Two modes. ExactKV has two integration points that share the code of §IV. In *tier mode* (the codec in the tiers), vLLM keeps raw KV and its stock FA3 backend; LMCache's cache server encodes each chunk it stores and decodes each chunk it loads, so DRAM and NVMe hold compressed records and vLLM's outputs are bit-identical to raw storage. Tier mode needs no change to vLLM and leaves LMCache's tier management in place (§V). In *pool mode* (the codec in attention), a vLLM attention backend keeps the engine's own pool compressed and attends over it in place (§VI); it adds GPU capacity, which tier mode cannot. Both use the same 3-bit primary exponent code; escape levels and record layout differ, as pool mode lays its records out for the attention kernel (§VI-A), so in pool mode the cache server moves each closed block as an opaque blob and runs no codec (§VI-E).

Units. A *chunk* is the KV of one 128-token block across all layers: 12 MiB raw for Qwen3-30B-A3B in BF16, 6 MiB per rank at TP 2. LMCache keys chunks by a rolling hash of the token prefix. A chunk is split into *planes* (the K or V of one layer) and each plane into 256-scalar *tiles*. The codec turns a chunk into one *record*; LMCache stores the record as one object in L_1 and L_2.

Store and load paths. After the step that completes a chunk, vLLM's connector asks the cache server to store it. The server encodes the chunk straight from the engine's blocks into a slot of a GPU store pool and returns; a store worker of its own then reserves an L_1 object of each record's length, copies the record into it and commits it (A, B); LMCache writes the object through to L_2 (C). On a lookup, LMCache returns the longest prefix it holds and prefetches L_2-only chunks into L_1 (1, 2). The load copies the records into GPU record buffers on a stream of its own and decodes them straight into the engine's blocks (3, 4). The forward pass never waits for a store, and no load path step reads a length back to the host.

IV. Paged-Native Lossless Codec

A. Record format

Fig. 4 shows the layout. The low byte of each scalar is stored verbatim. The high byte is looked up in a static per-head codebook of 14 values: a 3-bit primary symbol selects one of the seven most frequent values; code 7 escapes to a 3-bit secondary symbol that selects one of the next seven; secondary code 7 escapes to a raw byte, a two-level form of PFOR's patched exceptions [60]. Within a plane, the low bytes, primary symbols, secondary symbols and raw bytes form four streams, as in Parquet's byte-stream-split encoding [61]. A tile index gives, for every tile, the count of secondary and raw symbols before it, so a decoder can start at any tile without scanning its predecessors (R2): per group of eight tiles, two 32-bit counts and seven pairs of 16-bit offsets (36 B). Each chunk becomes one record: a 4 KiB header (version, mode, per-plane stream offsets), the plane payloads and padding to 4 KiB for O_DIRECT IO. A chunk that would not fit its raw envelope is stored raw, so the worst case equals raw storage.

The format spends $8 + 3 + 3p_e + 8p_r + 0.14$ bits per scalar, where p_e and p_r are the secondary and raw escape rates and 0.14 bits are the tile index. With calibrated codebooks on Qwen3-30B-A3B, $p_e = 0.061$ and $p_r = 0.0019$: 11.34 bits, or 0.709 of raw, as measured.

Why 3+3 bits. On the dump of §II-B, with per-plane tables and every escape's payload counted (Fig. 2c), the high byte costs 3.49 bits with one 3-bit level whose escapes store a raw byte ($3 + 8p_e$); 3.40 with ZipServ's window of seven exponents [35]; 3.61 with ALP-RD's [42] 3-bit dictionary of eight values whose exceptions store the byte and a 10-bit position in a 1024-value vector; 4.00 with one 4-bit level (as in SplitZip [5]); 3.24 for 2+3 or 3+4 and 3.20 for 3+3, against an order-0 entropy of 2.85. A per-plane Huffman code would recover about 0.02 of raw but give each scalar a data-dependent bit position, so a lane could no longer take its eight symbols from one 24-bit word in MMA-fragment or PRMT order, and decoding within a tile would serialize.

Codebooks are fixed at startup: a generic table, or one calibrated offline per model from one forward pass (1024 tokens suffice, Table I) and bound to the model by a digest.

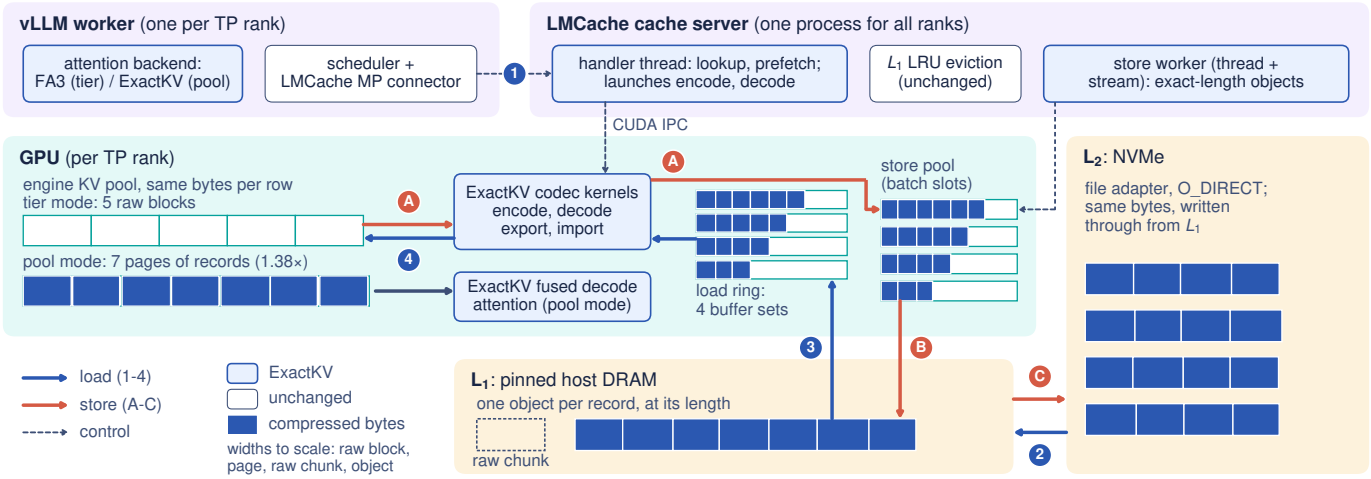


Fig. 3. ExactKV in vLLM and LMCache; indigo-edged boxes are ExactKV's, widths are to scale. Load path (indigo, 1–4; dashed: control): 1 lookup in the cache server, which prefetches L_2-only chunks; 2 L_2 to L_1; 3 copy of the records into one of four buffer sets; 4 decode straight into the engine's blocks (tier mode) or import of attention records (pool mode). Coral, store path: A encode from the engine's blocks into a store-pool slot after the producing step; B the store worker reserves one L_1 object per record and copies it out; C LMCache's write-through to L_2. Kernels run on the engine's GPU, launched by the server through CUDA IPC. Only compressed bytes cross PCIe and reach DRAM and NVMe.

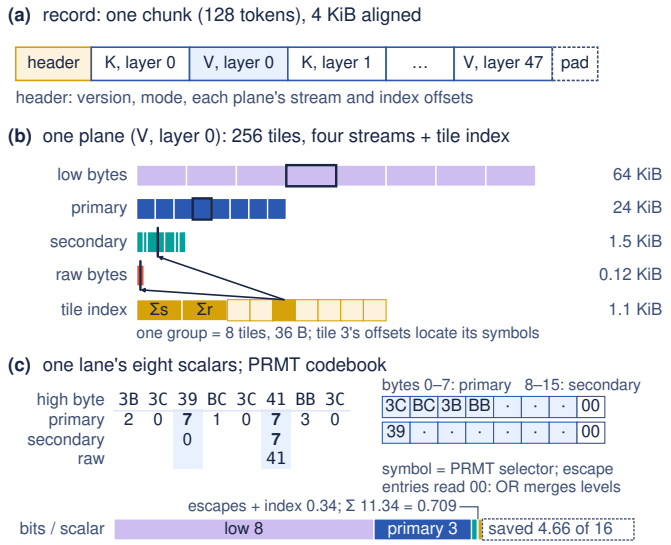


Fig. 4. Tier record layout (Qwen3-30B-A3B, calibrated codebooks, mean sizes). (a) A record: a 4 KiB header, then the K and V planes of every layer. (b) A plane splits into four streams and a tile index; one tile (outlined) has a fixed slice of the low-byte and primary streams and a variable one of the escape streams, which its tile-index group locates. (c) One lane's eight scalars; the 16-byte codebook a PRMT instruction indexes; bits per scalar, to scale.

TABLE I

STORED RATIO OF BF16 KV BY MODEL: 4096 TOKENS OF AGENT TEXT, CODEBOOKS CALIBRATED ON THE FIRST 1024 TOKENS AND MEASURED ON THE REST. TIER RECORDS WITH THE GENERIC AND THE CALIBRATED CODEBOOK, THE PER-PLANE ORDER-0 BOUND, ATTENTION RECORDS AND THE MEAN SIZE OF THEIR VARIABLE PART. $L{\times}H$: LAYERS × KV HEADS. OTHER MODELS' TIER RATIOS: PER-TILE INDEX LESS THE GROUPED INDEX'S 0.007.

		tier records			attention rec.	
Model	$L{\times}H$	generic	calib.	bound	ratio	var. B
Qwen3-4B	36×8	.729	.709	.676	.705	562
Qwen3-8B	36×8	.732	.709	.676	.705	561
Qwen3-32B	64×8	.731	.708	.674	.703	515
Qwen3-30B-A3B	48×4	.764	.709	.676	.704	550
Qwen2.5-72B	80×8	.722	.707	.672	.702	484
Llama-3-8B	32×8	.732	.707	.672	.702	479
Llama-3.1-8B	32×8	.729	.706	.672	.701	454
Llama-3.1-70B	80×8	.740	.705	.669	.700	395
GLM-4-9B-1M	40×4	.730	.707	.673	.702	459

Calibration lowers Qwen3-30B-A3B's ratio from 0.764 to 0.709; tables calibrated on agent text and on the benchmark's prompts differ by 0.0003. On nine Qwen, Llama and GLM models, 4B to 72B, dense and mixture-of-experts (MoE) (Table I), calibrated tier records store 0.705–0.709 of raw, 0.03 above the order-0 bound, and attention records (§VI-A) 0.70.

B. Fused decoder

One warp decodes one tile; each lane owns eight consecutive scalars. A lane spreads its 24-bit primary word to one nibble per symbol. An escape is the symbol 7 (all three bits set), so intersecting the nibbles with their one- and two-bit shifts marks escapes and a population count gives the lane's escape count without a per-scalar ballot. Two warp-wide exclusive scans plus the tile prefix give the lane's offsets into the secondary and raw streams. In Fig. 4 the lane marks scalars 2 and 5, takes two secondary symbols (0 and 7), and, since the second escapes again, one raw byte (41).

High bytes come from byte-permute (PRMT) instructions. Codebooks are repacked to 16 bytes per plane (primary entries in bytes 0–6, secondary in 8–14, escape entry zero), so a symbol nibble is directly a PRMT selector and one PRMT pair resolves a level for four scalars; one more PRMT, indexed by escape ranks, gathers escaped bytes, and an OR merges them because escape entries decode to zero. Decoding is branchless, since escapes are scattered across lanes. Values stay in registers and go out as 128-bit stores directly into each

destination block, reusing the registers when one chunk fills several blocks. A thread block (CTA) decodes 32 groups of eight tiles and prefetches the next group's words.

C. Fused encoder

The encoder runs in three launches with no host synchronization between them: a *count* stage reads the KV straight from the engine's paged blocks and scans per-tile escape counts, a single-warp *layout* stage assigns record offsets and decides raw fallback, and a *compact* stage writes the streams, merging secondary bits of different lanes with atomic adds on disjoint bit ranges. It runs on a store thread and stream behind the producer event of the step that wrote the KV; the thread synchronizes once per batch to read the stored lengths (R3), so the engine thread neither encodes nor waits.

D. Robustness

Every global write address of the decoder comes from host-validated metadata; record bytes only choose values, and reads they address are clamped to the staging extent, so a corrupt record restores wrong values into its own blocks and nothing else (checked by a GPU fuzz test that flips bits in descriptors, tile indices and payloads). The decoder waits on a model-stream event before writing blocks whose previous owner may still be read.

V. The Codec Inside LMCache

A tier's manager decides what to keep; ExactKV changes only what an object holds and how it moves between the GPU and L_1. LMCache's L_1 and L_2 management, LRU eviction, write-through and prefetch stay as they are. Three things change: an object's size, who produces it, and how a load turns it back into blocks. All of it is enabled by one server flag and off by default, which leaves LMCache's raw path untouched.

A. Exact-length objects

LMCache allocates every object of a registered cache from one layout, the raw chunk's. With the codec, the cache registers a flat byte layout whose size only bounds a record (its raw envelope), and each record reserves an L_1 object of its own length from the pinned pool. The file adapter writes the object as it is; records are padded to 4 KiB, so O_DIRECT still applies. A prefetch from L_2 needs an L_1 buffer of the stored size, which only L_2 knows: the adapter records each object's size when its store completes, and the prefetch controller reserves byte-layout objects at those sizes, one reservation per distinct size. Without this, a compressed tier saves PCIe bytes but no capacity. LMCache's pinned pool already serves variable sizes: first fit over an address-ordered free list, 4 KiB aligned, coalescing on free; its eviction loop counts bytes against the watermark and frees LRU keys. Records average 4.3 MiB per rank against 6 MiB raw and never exceed 0.84 of it, so freed extents are reused: at the same allocated bytes L_1 holds 1.39× as many compressed objects as raw ones, 1/0.716 for the * records (§VIII). InnoDB's table compression [54] rounds pages to power-of-two blocks, saving nothing at 0.71–0.84 of raw; its page compression punches holes at file-system blocks, as our 4 KiB alignment does. Fixed slots at 0.90 of raw (none overflowed) cut goodput at 640 GiB NVMe from 13.5 to 8.4 per minute (Table III).

B. Store path

A record's length is known only after encoding, and reading it synchronizes the GPU (R3). LMCache serves all stores and loads of one GPU on one handler thread, in order, so a synchronization in the store would delay every load queued behind it. The handler therefore only launches: it skips chunks already resident in L_1 (which LMCache would refuse to overwrite) and, for each batch of chunks (eight by default), encodes the engine's blocks straight into one slot of a pool of GPU buffers, records an event and returns; the engine may reuse its blocks once that event passes. A store worker, with its own thread and stream, waits on the event, reads the batch's lengths with one synchronization, reserves one L_1 object per record (records of equal length share a reservation), copies each record to its object and commits the batch. Encoding from the blocks replaces a gather into the pool, a copy into the codec's input and a copy out of its output: a batch costs five kernel launches instead of 22 and two passes over the raw bytes fewer. A full pool blocks the handler, so stores slow down rather than being dropped when encoding falls behind. A per-request tally reports the stored count and bytes once every batch has committed, so LMCache's accounting is unchanged.

C. Load path

A load names a prefix of chunks whose objects LMCache has pinned in L_1. Per batch, ExactKV copies each record into one of the codec's record buffers and decodes the batch straight into the engine's blocks (§IV-B); chunks wholly inside the engine's own prefix-cache hit are skipped. Batches with a raw record or a partly shared leading chunk decode into a scratch and are scattered with LMCache's transfer kernel, which handles vLLM's layouts. Record lengths come from the host-side object sizes, and the decoder's launch metadata depends only on the batch's length pattern, so it is uploaded once and reused: the load enqueues copies and kernels and never waits for the GPU. The copies run on a stream of their own and rotate through four sets of record buffers; each decode waits for its set's copies and each refill for that set's last decode. Copies of later batches thus run ahead while earlier batches decode, and the decodes queue back to back. One server thread handles every store and load of a GPU, so any wait there stalls all requests behind it (§IX).

Layouts and parallelism. vLLM 0.30's FA3 cache interleaves K and V per head (`[blocks, tokens, heads, 2×head]`); the codec views a block's K and V as separate planes by stride. Under tensor parallelism every rank registers its own cache and LMCache keys objects per rank; each rank's records have their own lengths. BF16 caches and vLLM's FP8 E4M3 caches (stored as bytes) use the same code with their own codebooks; an FP8 scalar's three mantissa bits are stored verbatim and its sign and exponent less a channel offset are coded (0.854 of raw).

VI. Compressed KV in GPU Memory

Tier mode saves capacity outside the GPU, but the engine's pool, which decides how many long requests run at once, still holds raw KV. Pool mode keeps the pool itself compressed, through an attention backend registered with vLLM's plugin entry point (`--attention-backend CUSTOM`), without changes to vLLM's source. vLLM still allocates blocks, builds block tables and schedules requests, but each block's per-layer page holds *attention records*.

A. FragPack attention records

A *FragPack* record, named for storing scalars in fragment order, is the K or V of one block, layer and KV head: 128 tokens by 128 dimensions, 32 KiB raw. High bytes use the 3-bit primary code of tier records with a per-(layer, K/V, head) table of seven values; an escape takes a 4-bit nibble selecting one of 15 further values, or 15 for a raw scalar. The record differs from a tier record in scalar order: the decode kernel (§VI-D) runs 256 threads per record, each consuming 64 scalars as operand fragments of tensor-core matrix multiply-accumulate (MMA), and the record stores scalars in exactly that order, so each thread reads its own bytes with 16-byte loads and no index arithmetic. A fixed *dense part* of 22 528 B holds the low bytes and primary symbols, chunk-major so a warp load covers 512 contiguous bytes; a *variable part* of at most 2 KiB holds a 48-byte header of per-warp escape and raw prefix counts, the escape nibbles and the raw scalars. With calibrated tables the variable part averages 550 B on agent KV, so a record is 0.70 of raw (Table I: 395–562 B on nine grouped-query attention (GQA) models). A record whose variable part exceeds 2 KiB, or with more than 32 raw scalars in a warp, is stored raw and read by the same kernel. FP8 records store mantissas verbatim and code sign and exponent less a per-channel offset (added back on decode), so channels at different scales share the head's primary values; offsets are fitted for K over positions up to 256K as RoPE rotates its channel pairs. On held-out agent KV at positions up to 240K the records are 0.84 of raw.

B. Pages and packing

A block's page for one layer is sized for its $2H$ records at an average variable part of 1280 B: 190 464 B for $H = 4$ against 262 144 B raw, so vLLM's KV budget yields 1.376× as many blocks (1.31–1.34× after the backend's buffers); the backend reports this size through its KV-spec hook. Records pack into the page in encode order (Fig. 5). The encoder runs one CTA per record, sizes the variable part v with block-wide scans and claims $22528 + v$ bytes by a compare-and-swap on the page's fill counter; a failed claim leaves the counter unchanged, so a smaller record of the block can still fit, and a record that does not fit goes raw to an overflow pool of 32 KiB slots, a bitmap allocated with atomic OR so encodes and imports on different streams can allocate concurrently. On agent KV, a 1280 B share spilled no record and a 1024 B share about 2%, each costing 32 KiB; fixed 24 KiB record slots (2 KiB for the variable part) would give 1.33×, against 1.38× with shared

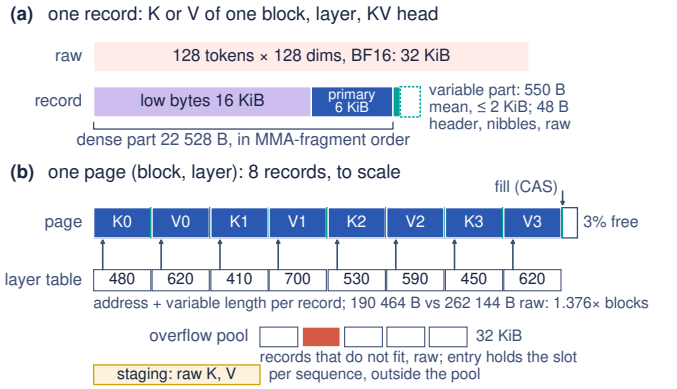


Fig. 5. The compressed GPU pool, to scale. (a) An attention record: a dense part of fixed size and a variable part (header of per-warp counts, escape nibbles, raw scalars). (b) A page (one block and layer) holds the $2H$ records in encode order, leaving 3% slack; the per-layer table gives each record's address and variable length. A record that does not fit goes raw to a 32 KiB overflow slot (none did on agent KV). Open blocks stage raw K and V per sequence, outside the pool.

pages and 1.42× for an ideal heap. Per-layer tables give every record's address and variable length, so kernels never compute positions inside a page.

C. Per-step state machine

Only complete blocks are encoded; a block still filling keeps raw records in a staging slot. Before each step the backend releases the staging of blocks no request still writes, claims staging for blocks that receive their first token, and lists those that receive their 128th; each layer writes new K and V into staging, attends through the tables and encodes the listed blocks. A running request has at most one open block, so one slot per schedulable sequence suffices. vLLM does not tell backends when blocks are freed, so the plugin mirrors each request's block list from the scheduler output and treats as open exactly the block holding its first uncomputed token (a superset exhausted staging at 32 requests). Every launch depends only on the padded batch shape, so decode-only steps replay as full CUDA graphs.

D. Fused decode attention

Decode is split along the context: a CTA of 256 threads handles one request, KV head and page range. Once per step, the device picks one range length for all requests, the shortest whose CTAs fit in one wave, so each request gets CTAs in proportion to its context. TMA copies each page's records (dense part plus actual variable length) a page ahead. Each thread decodes its 64 K scalars with byte-permute lookups into a register-resident table, patching escapes located by a warp scan over the header's prefix counts, into the A fragments of an `m16n8k16` MMA; computing $S^\top = KQ^\top$ lets a GQA group's eight query heads fill $n = 8$. The kernel reads about 0.70 of the raw bytes. Raw records skip the decode step, with bitwise identical output. Prefill is compute-bound and runs FlashAttention-3 unchanged on a BF16 workspace into which

the context pages are first decoded; fusing that step could save at most its 3% (§VIII-B).

E. Blobs in the external tiers

With attention records in the pool, the tiers need no codec. An export kernel copies a complete block's records of all layers ($2H$ per layer) into one *blob*: a 4 KiB-aligned head (version, geometry, a digest of the code tables, every record's variable length) followed by the records. An import kernel validates the head, packs the records into the destination page and only then swaps the record tables, so a failed layer keeps a consistent table; any failure marks the batch invalid and vLLM recomputes it. GPU, DRAM and NVMe then hold the same lossless bytes.

The cache server, not the worker, launches both kernels. The backend announces its page geometry through LMCache's layout hints when vLLM registers the KV cache, and shares its record tables over CUDA IPC before the first store or load. On a store, the server exports the request's closed blocks into the store worker's fixed-size GPU pool on its own stream and returns; the worker reads the blob lengths back and commits one exact-length object per block (§V-B), so the engine's thread never waits on an export. A block still open or already freed fails its export and is simply not stored. On a load, the server copies the blobs into a staging buffer, imports them into the destination blocks and synchronizes once per request; a failure fails the load and vLLM recomputes. Because an LMCache chunk must then be exactly one 128-token vLLM block, pool mode fixes LMCache's chunk size to the block size.

VII. Implementation

ExactKV is a branch of LMCache 0.5.6 that runs with unmodified vLLM 0.30: the codec package (4.9 kLoC of Python and TileLang kernels), its integration into the cache server (1.0 kLoC, §V) and the attention backend (2.6 kLoC of Python, 1.8 k of CUDA C++), with 2.3 kLoC of tests. Codec on or off, codebooks and the store batch are fixed at server startup; the raw path is LMCache's own. The cache-server change is about 160 lines of LMCache behind one flag and is not yet proposed upstream; tier mode needs no model requalification, pool mode that of one attention backend.

The attention backend is loaded as a vLLM plugin. Its 1.3 GiB of GPU memory (overflow pool, staging, prefill workspace) is charged to it in every AgentX comparison. Head size, GQA group and block size are compile-time parameters; we build for BF16 and FP8 KV, 128-dimension heads, GQA groups of up to 8 (smaller ones padded) and 128-token blocks.

VIII. Evaluation

Q1 compares the kernels with existing codecs and FlashAttention-3; **Q2** asks what compressed tiers buy (tier mode), **Q3** what a compressed GPU pool buys once tiers hold every working set (pool mode), and **Q4** what compression costs and what is exact.

A. Setup

Hardware. Two 48-core Xeon Platinum 8558, 1 TiB DRAM, four 3.84 TB Memblaze P6531 NVMe drives in one btrfs volume (single data profile). The server is a shared lab machine; we use two of its H800 SXM modules (80 GB, 700 W), which this chassis connects over PCIe without NVLink. No other job used the two GPUs during a run. **Models.** Qwen3-30B-A3B-Instruct, a mixture-of-experts (MoE) model with 3B active parameters (FP8 weights, BF16 KV, 96 KiB per token), at TP 2, unless noted; GLM-4-9B-Chat-1M, a dense model of another family (40 layers, 4 KV heads, 80 KiB per token, BF16 weights; 1.42M tokens in 54 GiB of KV per GPU), at TP 2 as a second AgentX model; Qwen3-8B and Llama-3.1-8B (Hermes-3 weights [46]) for kernel and ratio generality. vLLM 0.30 with its V2 model runner, asynchronous scheduling, asynchronous KV loading, 8192-token chunked prefill and 128-token blocks.

LMCache deployment. Every LMCache arm runs LMCache 0.5.6's cache server (multiprocess mode, LMCache-driven transfers), raw or with the codec, and differs from the others only in the flags we name. L_1 is 400 GiB of pinned DRAM, the GPU-proportional share of host DRAM that AgentX's harness grants two of our five GPUs, allocated eagerly (no lazy growth). L_2 is LMCache's native file adapter with O_DIRECT and 16 IO workers on a directory with copy-on-write disabled; it loads 3 MiB objects at 6.6–7.4 GB/s and stores at 4.6 GB/s. Buffered IO would add the page cache, a DRAM tier outside the budget. L_1 evicts LRU objects once it is 90% full, 5% of its keys at a time; LMCache's defaults (80% and 20%) would leave a fifth of DRAM unused between its once-a-second eviction passes. A run starts only after the server reports its whole L_1 pinned and every L_2 adapter healthy (2–3 min for 400 GiB), so no arm loses capacity to initialization. Tier records store 0.709 of BF16 and 0.854 of FP8; arms marked * ran before the grouped tile index (§IV), at 0.716 and 0.868; a current-format rerun at 640 GiB is within 0.2%. **Baselines.** LMCache raw with the same budgets; LMCache with its FP8 L_2 serializer (lossy); vLLM with FP8 KV, which halves every level and changes the values, with raw LMCache tiers. We run LMCache's serializer with its stock settings (one CPU worker). We do not port the codec to Mooncake or SGLang HiCache; §X simulates how the replacement policy moves the cliff. CacheGen and TurboQuant [73] are lossy, and CacheGen is not in LMCache's multiprocess server (§XI).

B. Q1: Kernel microbenchmarks

Table II compares codecs on the same card and KV, every restore checked byte-exact. Because ExactKV writes restored values from registers into the destination blocks, it restores into a paged cache at 81% of a contiguous D2D copy, while reading only 0.709 of the bytes; the raw-gather row (PyTorch's `index_copy_` of whole block rows, 256 blocks per call, not a tuned gather kernel) is 2.2× slower. A contiguous codec must decode and then scatter: SplitZip's decode reaches 83% of a D2D copy, and ExactKV is 3.3× faster than SplitZip plus our gather and still 1.8× faster if the scatter ran at copy speed,

TABLE II

CODECS ON ONE H800 SXM AND THE SAME QWEN3-30B-A3B KV (3.2 GB, 256 CHUNKS PER CALL), RAW-EQUIVALENT GB/S. "PAGED" RESTORES INTO SCATTERED BLOCKS OF A PAGED POOL; "CONTIG." STOPS AT THE CODEC'S OWN OUTPUT BUFFER.

Codec	ratio	enc	paged	contig.
Device copy (raw)	1.000	–	–	1523
Raw gather to paged	1.000	–	547	–
ExactKV	**0.709**	**528**	**1229**	–
SplitZip [5]	0.768	326	376	1264
ExANS [44][†]	0.698	7	186	–
nvCOMP ANS	0.725	113	181	276
nvCOMP Bitcomp	0.876	116	152	212
nvCOMP LZ4	1.000	9	146	202
ZipServ code[‡]	0.722	–	–	–
ZipFusion blocks[§]	0.79	–	–	–

[†]Gathers and scatters 16 KiB pages itself; we batch its header readback into one copy (78 GB/s as shipped). [‡]ZipServ's TCA-TBE code [35] computed offline, window per plane, with ExactKV's tile index (0.713 without it); ZipServ ships a weight compressor and GEMM only. [§]ZipFusion's format [33] computed offline: 512-value blocks with a 12-bit code, raw when a block spans more than 16 exponents (K 0.81, V 0.77); its kernel targets FlashInfer and has no tier path.

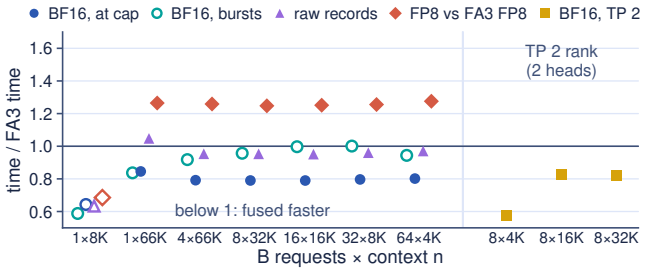


Fig. 6. Fused decode attention time over FlashAttention-3's on one layer of Qwen3-30B-A3B KV (4 KV heads, one H800 SXM), by batch: BF16 back to back, which holds the card at its 700 W cap; BF16 in idle-gapped bursts (median of three); the fused kernel over raw-stored records; FP8 records against FA3 FP8; and one TP 2 rank's two heads. Hollow: below the power cap. Below 1: fused faster.

while storing 8% fewer bytes. ExANS stores 1.6% fewer bytes (0.698, between the global and per-plane order-0 bounds of §II-B), but restores 6.6× and encodes 75× slower.

Fused decode attention. Fig. 6 interleaves the kernels in one process, each called back to back. Batches hold the card at its 700 W cap, where time follows energy: FA3 takes 1.18–1.27× the fused kernel's time and energy (1.18 for one 66K context; 1.21–1.22× for a TP 2 rank's two heads at 16K and 32K, 1.74× at 4K, where the fused kernel holds 1762 MHz), and raw-stored records take 1.20–1.24× longer, so reading 0.70 of the bytes pays for decoding. Followed by a memory-bound 320 μs GEMM sharing its clock, each call still saves 52–58 μs; in serving, fused-pool arms clock 7–20% higher at the same power. One 8K context stays below the cap: its 1.6× (FA3 caps it at 32 splits) ties in the step, and batches below the cap take 0.83–1.00× FA3's time; FP8 is slower (§X). Prefill over the decoded workspace costs 1.1–2.9% more than FA3 (2K–8K chunks).

C. Q2: Step 1, tier capacity on AgentX

We run AgentX [20] with its official harness (agentx-harness 1.0.7, dataset `cc_traces_weka_062126_256k`, seed 42). It replays the 393 Claude Code sessions of cc-traces [47] as session trees with subagents and recorded think times. Each client joins a session 25–75% of the way in, sends primers that rebuild its live contexts and ten warmup requests; all metrics cover only the profiling window that follows. Below the cliff the cached share still falls inside it (raw on 640 GiB: 0.88 to 0.17 by 5-minute bin, Fig. 8b), so window averages favor those arms. A token-0 marker, required by the AgentX scenario, keeps sessions from sharing a prefix, so two clients replaying the same trace session do not hit each other's KV. The harness caps DRAM offload at the GPU-proportional share of host DRAM, 400 GiB for 2 of our 5 GPUs. Prompts reach 254K tokens (p50 89K), so we serve Qwen3-30B-A3B with TP 2 on two H800s and fill each GPU: 14.6 GiB of weights and 54 GiB of KV (1.18 M tokens). The attention backend charges its 1.3 GiB to the same 54 GiB; the cache server's CUDA context and the codec's GPU buffers (1.0 GiB per GPU with the codec) live outside vLLM's budget in every LMCache arm.

Admission. The harness's concurrency counts live root sessions; subagent requests take no slot. Open loop, it ties the working set to the requests in flight: at 16 clients every tier holds the working set (0.95 cached), yet vLLM queues each request for 19–23 s. To grow the working set past DRAM without overloading the engine, a 140-line FIFO proxy admits S requests at a time, as `vllm bench serve --max-concurrency` does. S = 8 is the larger of 8 and 12 at which the stock pool keeps vLLM's waiting queue near empty (0.1–0.6 requests; S = 12 leaves 1.5–1.7), which favors the stock pool (§VIII-D). Harness TTFT is then mostly the proxy wait (190–354 s at P = 64, as Little's law gives), so we report engine TTFT (harness TTFT minus the proxy wait) and goodput: requests per minute with engine TTFT $\leq$ 2 s and ITL $\leq$ 40 ms. Other users' jobs could use the host's CPUs and other GPUs; repeated runs show the spread.

Sizing. With admission fixed, the number of live sessions P sets the reused working set (a session's KV is live from its first to its last request): mean 306, 380, 489, 584 and 641 GiB raw at P = 32 to 64 in steps of 8 (p90 755 GiB at 64). Slower arms finish shorter prompts in the window, so we give request rates next to token rates. Most runs use P = 64 with 400 GiB of DRAM and vary the NVMe tier from 400 GiB to 2 TiB, which holds every set; each tier runs its own LRU. Deployments with more KV per token (Llama-3.1-70B: 3.3× ours) or several engines per node reach these ratios with larger tiers. The profiling window is 1800 s; repeated runs differ by 0.2–13.5% in throughput (35% for raw FP8 KV, DRAM only); tables and figures mark repeated points. Warmup takes 10–28 minutes for arms that hold the working set, 88–99 at the cliff and 42–143 below it.

Capacity equivalence. Table III. With NVMe holding every working set, tier mode keeps raw LMCache's cached share (0.94), ITL and goodput (15.4 vs 15.1 per minute) and costs

TABLE III

AGENTX, $S = 8$, 400 GiB DRAM; $P = 64$ UNLESS STATED. INPUT THROUGHPUT, ENGINE TTFT, ITL (MS PER OUTPUT TOKEN), THE SHARE OF PROMPT TOKENS SERVED FROM A CACHE (VLLM'S GPU PREFIX CACHE OR LMCACHE) AND GOODPUT (REQUESTS PER MINUTE WITH ENGINE TTFT ≤2 S AND ITL ≤40 MS), OVER THE PROFILING WINDOW. RAW CONTROLS SCALE NVME OR BOTH TIERS BY 1/0.716, OR MOVE 320 GIB TO DRAM (ABOVE THE HARNESS'S DRAM CAP). P99: NEAREST RANK; RATIO RANGES IN THE TEXT INCLUDE REPEATED RUNS. ^{r}: REPEATED. *: RECORDS AT 0.716 (FP8 0.868), §VIII.

NVMe	arm	k tok/s	TTFT (s) mean	p50	p99	ITL	hit	good
2 TiB	raw^{r}	27.5	1.69	0.99	18.6	26.0	.94	15.1
	tier mode*	26.9	1.85	0.87	26.8	26.8	.94	15.4
	pool mode	28.8	1.34	0.96	5.3	25.6	.94	16.2
1200 GiB	raw	26.1	1.97	1.03	17.3	26.6	.93	14.0
800 GiB	raw^{r}	20.0	3.78	1.13	52.7	32.6	.84	9.4
	tier mode*	25.0	1.98	0.90	23.8	26.5	.92	14.4
640 GiB	raw^{r}	12.2	6.76	1.11	78.7	43.5	.55	4.6
	tier mode^{r*}	24.6	2.34	0.88	38.0	26.9	.91	13.5
	tier, fixed slots	19.5	3.92	1.16	34.2	35.7	.83	8.4
	pool mode	25.5	1.76	0.93	18.8	27.0	.90	13.4
	FP8 serializer	26.9	2.28	1.32	18.4	26.4	.93	12.7
	FP8 KV, raw	33.0	1.06	0.72	6.6	21.6	.94	20.6
	FP8 KV, tier*	33.1	1.02	0.70	4.8	21.7	.94	20.6
480 GiB	raw	11.1	7.66	1.45	73.0	53.9	.44	3.5
	tier mode^{r*}	21.3	3.44	0.97	43.8	31.3	.85	10.5
400 GiB	tier mode*	12.6	6.67	1.16	72.0	50.2	.58	4.4
DRAM only, $P = 40$								
	raw^{r}	11.3	7.33	1.91	48.4	62.0	.45	2.9
	tier mode*	25.1	1.56	0.80	15.7	30.7	.91	12.5
	FP8 KV, raw	34.1	1.17	0.76	8.2	22.6	.94	19.2
DRAM only, FP8 KV, $P = 64$								
	raw^{r}	18.8	3.59	0.83	31.1	36.3	.68	8.0
	tier mode*	32.2	1.13	0.66	11.6	22.0	.92	19.4
Raw controls, DRAM + NVMe GiB								
400+672	raw^{r}	14.9	5.86	1.31	73.8	45.2	.66	5.1
559+672	raw	20.9	3.38	0.85	45.4	32.2	.85	10.7
720+480	raw	20.3	3.20	0.84	39.8	33.9	.82	9.9

2.2% of its throughput, against 1.1% between raw runs; its median TTFT is 12% lower, since loads move 0.716 of the bytes, and its p99 higher (26.8 vs 18.6 s, set by the six slowest of about 580 requests). Below that, the codec's results follow DRAM plus NVMe in raw-equivalent bytes (Fig. 7a). Tier mode on 400 GiB of DRAM and 480 of NVMe (1229 raw-equivalent GiB) matches a raw control with 559 and 672 (1231, one run): 21.3–22.6 vs 20.9 k tokens/s, 0.85 cached each, 10.5 vs 10.7 requests per minute, and cached shares within 0.08 in every 5-minute bin. Coarser points are consistent with it: tier mode on 880 GiB in total has raw's goodput on 1200 (10.5 vs 9.4), and on 1040 nearly raw's on 1600 (13.5 vs 14.0). The cliff alone bounds the capacity factor between 1.22 and 1.50. Above the cliff the curves meet: at 1676 raw-equivalent GiB tier mode has raw's goodput at 1600 (14.4 vs 14.0) but 4% less throughput (codec cost and run spread). Near the cliff a run either recovers to 0.9 cached or decays toward 0.15, and raw-equivalent bytes, not their DRAM share, decide which: every run at 1117 raw-equivalent GiB or less decays (raw on 672, tier mode on 400: 0.66 and 0.58 over the window), and every run at 1200 or more recovers (raw on 800, 720+480 and 559+672; tier mode on 480 and 640), one

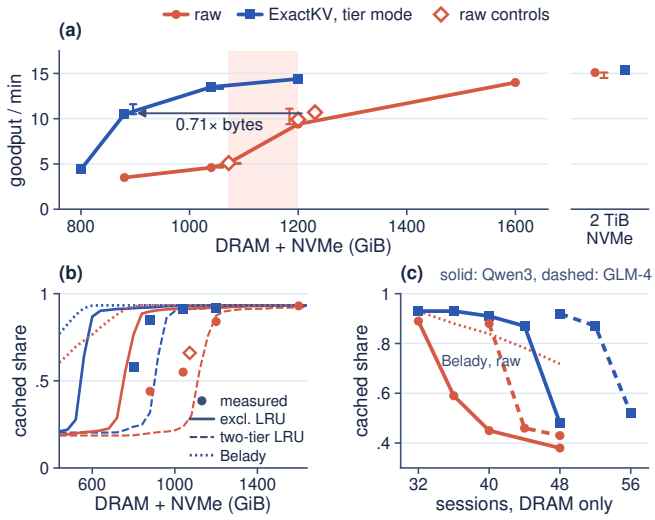


Fig. 7. A compressed tier acts as a raw tier 1.4× its bytes (AgentX, 400 GiB DRAM, 64 sessions). (a) Goodput by physical DRAM plus NVMe; shaded: raw's cliff; arrow: the matched pair; bars beside a point: repeated runs; hollow diamonds: raw controls that scale or move tier bytes (Table III); right: 2 TiB of NVMe. (b) Cached share of prompt tokens, measured and simulated on the 2 TiB arm's request stream (one exclusive cache of the same bytes, and LMCache's two tiers; compressed at 0.709; other streams: §X); the simulation omits vLLM's GPU prefix cache. (c) DRAM only, by live sessions; solid: Qwen3-30B-A3B, dashed: GLM-4-9B; dotted: Belady on raw bytes, each arm's own stream.

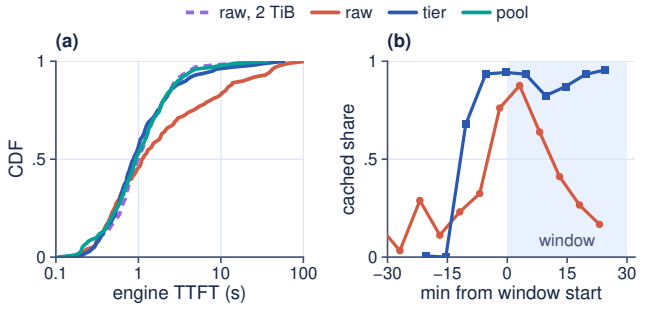


Fig. 8. AgentX, 400 GiB DRAM + 640 GiB NVMe. (a) Engine TTFT over the profiling window (raw, tier and pool mode; dashed: raw on 2 TiB). (b) Cached share of prompt tokens by 5-minute bin of request completion, raw and tier mode only, aligned at each arm's window start (tint: the window). Every request in the window got a first token. CDFs use the harness's request records and interpolate, so tails differ from Table III's.

or two runs per point. The window opens just after the primers re-warm every session, and within it neither branch reverses. On 672 GiB of NVMe, 400 GiB of DRAM decays and 559 recovers: despite write-through, DRAM adds capacity.

On fixed tiers. The same equivalence moves a fixed deployment along a steep curve. With 640 GiB of NVMe, raw LMCache falls off the cliff of Fig. 1: each session's prefix is evicted shortly before the session returns, so the engine recomputes 0.45 of prompt tokens, one 8K-token chunk at a time, and recomputed chunks in every batch raise ITL to 43.5 ms. Its TTFT splits in two (Fig. 8a): half the requests start within 1.1 s, as on 2 TiB, while the slowest quarter recompute

and take 4.6 s or more. With the same tiers holding compressed records, 0.91 of prompt tokens stay cached (0.94 against raw's 0.17 over the window's last 10 minutes); tier mode has 2.8–2.9× raw's goodput (3.0–3.3× on 480 GiB), p90 TTFT 3.8 instead of 18.9 s and 1.85–2.0× its input tokens (1.8–1.9× its requests), and warms up in 23 minutes. These multipliers measure LMCache's write-through LRU at its cliff with the codec on top; §X separates the two. The multiplier does not hinge on the 2 s target: with engine TTFT ≤ 1, 5 or 10 s it is 2.6, 2.9 and 2.8×, and 2.4× without the ITL bound. It is 2.1× over the window's first half; in the second every arm loses goodput to longer prompts (2 TiB raw: 20.9 to 9.3 per minute) and raw serves none within target. Away from the cliff the gain is smaller (tier mode on 640 against raw on 800: 1.08–1.23× the tokens); on 400 tier mode itself falls off. GLM-4-9B, with 0.83 of the KV, flips one size down: on 400 GiB tier mode caches 0.90 against raw's 0.44 and serves 4.0× its goodput.

More sessions on the same DRAM. Without NVMe the cliff sits at the DRAM tier (360 GiB at LRU's 0.9 watermark), and the number of live sessions P sets the working set (Fig. 7c). Raw LMCache holds 32 sessions (306 GiB live mean) and falls by 36; tier mode holds 44 and falls at 48, and at 36 and 40 sessions serves 3.5 and 4.3× raw's goodput. On GLM-4-9B raw falls by 44 and tier mode by 56 (7.9× at 48). The cliff moves by 1.2–1.5× in sessions, around the stored ratio's 1.4×; the multipliers are measured between the two cliffs. With vLLM's FP8 KV raw holds 56 (0.90) and falls at 64 (0.61–0.75 cached), where tier mode caches 0.92; at 72 FP8 records (0.868) still cache 0.80 against 0.58.

Cold and warm passes. When every request hits, compression decides which tier serves the hit. The repeat trace sends 48 prompts of 8K–128K tokens (one in flight, 64 output tokens) and then the same 48 again (261 GiB of raw KV, 188 compressed). With 400 GiB of DRAM every hit loads from L_1 and compression changes warm TTFT by 0–11%. With 220 GiB the compressed set fits in DRAM while part of raw's hits load from NVMe, and compression cuts warm TTFT 1.61–1.70×; with 160 GiB, below both, 1.26–1.42× (pool mode 1.17–1.60×), through fewer NVMe loads of 28% fewer bytes. LMCache's FP8 serializer casts loads to BF16 on the CPU (23 GB/s), so its warm TTFT is 1.14–1.47× raw's. Cold TTFT stays within 2% of raw's. GLM-4-9B (DRAM scaled by its KV size) repeats the pattern (1.33–1.69× below DRAM).

Shared long contexts. SCBench [48] asks follow-up queries over one long context. We replay its eleven English tasks as 887 AIPerf sessions (4–8 turns, 112K-token contexts, think times from WildChat [49]) on 128 GiB of DRAM; records store 0.710 of raw. With 32 sessions tier mode caches 0.45 against 0.29, serves 1.15× the input tokens and halves mean engine TTFT (17.0 vs 35.2 s); with 16, 0.69 against 0.58, and median TTFT falls from 3.8 to 0.5 s.

Lossless versus lossy. Lossy formats store fewer bytes and win on throughput where accuracy permits. LMCache's FP8 serializer stores half the NVMe bytes and on 640 GiB serves 9% more input tokens than tier mode, with 50% higher median TTFT (loads are cast to BF16 on the CPU) and lower p99 (18.4 vs 38.0 s). vLLM's FP8 KV halves every level, the GPU pool included, and on 640 GiB serves 1.34× tier mode's input tokens (goodput 20.6 vs 13.5). Both change outputs (§VIII-E). The codec composes with FP8 KV: where FP8 KV still overflows DRAM (P = 64), tier mode raises its goodput from 6.6–10.6 to 19.4 per minute (1.8–2.9×).

TABLE IV
AGENTX WITH NVME HOLDING EVERY WORKING SET (P = 64, 400 GIB DRAM, 2 TIB NVME), AT TWO GPU KV BUDGETS PER GPU AND S = 8 UNLESS STATED. POOL: KV TOKENS THE ENGINE HOLDS; WAIT: MEAN REQUESTS WAITING IN VLLM; HIT, GOOD: AS IN TABLE III. BOUND: TIER MODE WITH THE RAW POOL ENLARGED TO 37.4 GIB, ABOUT THE COMPRESSED POOL'S TOKEN COUNT. FP8: VLLM'S FP8 KV, RAW LMCACHE TIERS OR FP8 POOL MODE. ^{r}: REPEATED RUNS, WITHIN 2%. *: AS IN TABLE III.

				TTFT (s)						
KV	arm	pool	k tok/s	mean	p50	p99	ITL	wait	hit	good
54 GiB	raw, S 12	1.18M	28.7	5.16	1.52	43.5	34.3	1.5	.94	10.6
	pool, S 12	1.59M	31.1	1.94	1.07	14.9	35.6	0.4	.94	14.9
	raw, S 16	1.18M	28.5	13.3	9.06	87.3	38.0	4.4	.94	3.7
	pool, S 16	1.59M	32.5	4.62	1.65	45.6	41.6	1.5	.94	6.8
28 GiB	raw	0.61M	engine stalled (see text)							
	tier*	0.61M	24.7	9.43	3.25	70.3	22.1	2.6	.94	7.2
	pool	0.80M	27.4	4.91	1.40	51.6	22.4	1.3	.94	11.3
	bound^{r*}	0.82M	25.7	5.09	1.12	51.5	24.3	1.3	.94	11.5
	FP8, S 12	1.22M	35.4	3.45	1.10	31.3	27.7	1.1	.94	16.1
	FP8 pool, S 12	1.29M	31.5	3.00	1.15	34.1	33.8	0.8	.94	15.2

Step 1. A compressed tier acts as a raw tier 1.4× its bytes: the same DRAM and NVMe hold more sessions, or the same sessions need about 0.71 of them, and hits move to faster tiers. The gain is largest at LRU's cliff, where that capacity decides whether a run recovers or decays.

D. Q3: Step 2, the GPU pool

Pool mode keeps vLLM's pool in FragPack records, so a KV budget holds 1.31–1.34× the tokens after the backend's 1.3 GiB of buffers. With NVMe holding every working set, every arm serves 0.94 of prompt tokens from a cache, so only the pool binds, and the gain comes from vLLM's queue, not from loads (Table IV).

More requests in flight. At 54 GiB per GPU and S = 8 the raw pool runs every admitted request (0.3 waiting); pool mode adds 4.8% throughput, cuts mean TTFT by 21% and p99 from 18.6 to 5.3 s (Table III). At S = 12 the raw pool runs 10.3 requests with 1.5 waiting and mean TTFT triples (5.16 s); the compressed pool runs 11.4 with 0.4 waiting, as raw at S = 8, and serves 13% more input tokens than raw at S = 8 at nearly its goodput (14.9 vs 15.1), with the higher ITL of larger batches (35.6 ms; raw at S = 12: 34.3). At S = 16 both queue and pool mode's ITL passes 40 ms. GLM-4-9B repeats the step one slot count up: its compressed pool at S = 16 has raw's queue and goodput at 12 with 18% more input tokens.

A smaller budget. At 28 GiB per GPU (26 GiB more weights), tier mode loses 8% of its throughput and 2.6 requests wait instead of 0.4. Pool mode's 0.80M tokens halve the wait: 10.9% more input tokens than tier mode, 1.57× its goodput

(11.3 vs 7.2 per minute), p99 52 instead of 70 s. Raw LMCache stalled twice: vLLM 0.30 reserves blocks only for loads in flight, and requests with finished loads held 72–82% of the pool; we count this as a vLLM limit of small pools, not a codec gain. FP8 KV doubles the raw pool (16.1 per minute at $S = 12$); FP8 pool mode adds 6% tokens but loses 11% throughput to its slower kernel, so FP8 KV pairs with raw pools or tier mode.

Capacity equivalence. The bound row gives tier mode a raw pool of 0.82M tokens (37.4 GiB), slightly more than the compressed pool's 0.80M and more than a real deployment has. It matches pool mode's wait (1.3), goodput (11.5 vs 11.3) and p99 (51.5 vs 51.6 s): the compressed pool acts as a raw pool of its token count. Pool mode serves 4.6–6.6% more input tokens than the bound, with 3.5% lower mean TTFT and 8% lower ITL (22.4 vs 24.3 ms). Measured directly (eight requests decoding at the power cap), the compressed pool's step is 1.3–5.6% shorter than FA3's at 8K–96K tokens per request, and 6.3–7.6% at 96K with LMCache attached and 8K prefills. The remainder over the bound also carries tier mode's 2.2% codec cost and the bound's 2% run spread. With both capacities short (640 GiB of NVMe), pool mode has tier mode's goodput (13.4 vs 13.5) with 51% lower p99 TTFT.

Step 2. A compressed pool acts as a raw pool 1.31–1.34× its budget without paying it back in decode time: it admits 12 long requests with the queue a raw pool has at 8 (16 at 12 on GLM-4-9B), serving 13–18% more input tokens at raw's goodput and 1.37× its ITL (35.6 ms, inside the 40 ms target).

E. Q4: Costs and exactness

Capacity to spare. When raw tiers already hold the working set, the codec buys nothing and costs little. With DRAM alone at $P = 24$ (0.94 cached in both arms), tier mode serves 28.5 vs 28.9 k tokens/s (−1.3%), mean TTFT 1.10 vs 1.15 s and the same goodput (18.1 vs 18.0 per minute); with 2 TiB NVMe it costs 2.2% (Q2). **Overhead.** Without reuse every request pays the store: 320 distinct 32K-token prompts (eight in flight, 3 GiB of KV each) take 0.3% longer in tier mode and 0.9% in pool mode; TTFT and ITL are within 1.7%.

Tier-only: bit-exact. The GPU test suite compares every restored block byte for byte with its source, including raw fallback, partial tiles, fan-out and corrupted records. End to end, every greedy output with compressed tiers equals the output with raw tiers: all 192 repeat-trace requests at 220 and 160 GiB of DRAM and all 288 of GLM-4-9B's, cold and warm (SHA-1 of the text). Raw's own warm and cold passes differ in 8 of 48 in every arm (a prefill over a loaded prefix sums in another order); tier mode reproduces raw's warm and cold outputs exactly, while warm passes with LMCache's FP8 serializer differ from warm raw passes in 5 of 48. On RULER through LMCache (40 prompts per task and length, 88% of warm keys from NVMe), warm FP8-serializer passes differ from warm raw ones in 2 of 120 multi-key and 19 of 120 variable-tracking answers, with scores within 1.5 standard errors. The case for exact tiers is operational: compression never changes an output, whichever tier served its prefix, so a tier change needs no requalification.

Pool mode: lossless storage, another kernel. Outputs over coded and raw records are bitwise equal. The kernel sums in a different order than FA3 with the same error: against an FP64 reference at 32K and 128K tokens its mean absolute decode error is 3.1e-5, as is FA3's; on FP8 KV it is 3.0e-5 against FA3 FP8's 5.1e-4, since FA3 also quantizes the query to FP8. GSM8K [59] (1319 questions, 5-shot) scores 0.885 with the fused kernel, 0.881 with FA3, 0.883 with vLLM's Triton backend and 0.881 with FP8 KV, within one standard error (0.009). On long-context tasks (Qwen3-30B-A3B, 50 prompts per cell at 32K, 64K and 128K tokens), RULER [58] multi-key needles score 0.993 with the fused kernel and FA3 and 0.953 with FP8 KV, which misses six of FA3's hits and gains none (exact McNemar $p = 0.03$, uncorrected, one of three tasks). Five-hop variable tracking, whose answers often hinge on whether to stop after the first name, scores 0.665 with FA3 and 0.658 fused on 450 prompts: 411 answers are identical, FA3 alone is exactly right on 15 and the fused kernel on 10 (McNemar $p = 0.42$), and the paired score difference is 0.007 ± 0.016 (95%). On another 150, FA3 scores 0.656 (a rerun is identical), Triton 0.624 ($p = 0.19$) and FP8 KV 0.661 ($p = 0.69$), changing 41 and 63 answers. FP8 KV runs as vLLM ships it: scale 1.0 (no calibrated scales in the checkpoint) and FA3's FP8 path in vLLM 0.30, which includes the two-level accumulation fix [74]. With attention sharpened to a few tokens, the fused kernel's outputs are correctly rounded as often as FA3's or more. Operators who must reproduce FA3's bits run tier-only.

IX. Lessons Learned

Measure capacity first. In an earlier tier store of ours, faster loads moved summed TTFT by a few percent; capacity near the cliff moves throughput 1.85–2.0×. Bytes per session and placement are levers that compose; only the codec leaves the tier's policy unchanged.

Compressed caching is a buffer-management problem. Three choices from database storage decide whether a ratio reaches capacity. Objects must be exact-length (§V). Records that die together should share a page with an overflow area, as slotted pages do: 1.38× against 1.33× for per-record slots (§VI). The layout should follow the consumer, as in FastLanes and PFOR: tier streams in one warp's decode order, attention records in MMA-fragment order.

Count launches, not bytes. After the store worker took encodes off LMCache's handler thread (§V-B), tier mode still lost 7%: the cache server's CUDA context is time-sliced against vLLM's, so every launch stretches decode steps. Five launches per batch instead of 22 cut the loss to 2.2%, which an idle-GPU benchmark hides.

X. Discussion and Limitations

Sizing rule. A tier's capacity in raw chunks is its bytes over the stored ratio, so the codec needs 0.71 of the DRAM and NVMe for the same working set (0.85 for FP8 KV), and

DRAM plus NVMe in raw-equivalent GiB orders the runs (Fig. 7a). For an operator: (1) estimate the live working set W (sum of KV of sessions between their first and last request); (2) under LMCache's write-through LRU with 400 GiB of DRAM and $P = 64$, raw tiers recovered between $1.74W$ and $1.87W$ of DRAM plus NVMe ($W = 641$ GiB, the mean) and came within 7% of unbounded tiers' goodput at $2.5W$; tier mode needs 0.71 of either. A single DRAM tier recovers at 1.1–$1.2W$; we have not isolated why two tiers need more; (3) turn on tier mode when hits come from NVMe or the cached share is visibly below that of unbounded tiers, and pool mode when vLLM's waiting queue is nonzero while at least 0.9 of prompt tokens hit. All three signals are counters vLLM and LMCache already export.

Policy dependence. We replay measured request streams through one exclusive cache (capacity scaled by $1/r$ for the codec) and LMCache's two write-through tiers (Fig. 7b,c). On each DRAM-only arm's own stream one LRU cache predicts the cached share within 0.076 (nine arms). Between the DRAM-only cliffs ($P = 36$–44) Belady's offline-optimal policy [79] on raw bytes caches 0.78–0.91 and ARC [75] and S3-FIFO [72] 0.20–0.73, against 0.81–0.92 for LRU over tier records, ahead by 0.01–0.05 on every stream: in one tier the gain is not a policy artifact. With NVMe it largely is. The two-tier model (DRAM hits do not refresh NVMe, as in LMCache) recovers only once NVMe alone holds about 800 raw-equivalent GiB, missing every run whose recovery needs DRAM (tier mode on 480, both controls). At 880 and 1040 GiB exclusive LRU [76] on raw bytes caches 0.86 and 0.89 on those arms' own streams, about tier mode's 0.85 and 0.91, so an exclusive tier manager alone would recover most of the gain there (with the codec, 0.92 instead of 0.77 at 800). These curves assume $1/r$ capacity and charge no migration IO.

Costs and limits. Tiers that already hold the working set gain nothing and pay 0–2.2% of throughput (§VIII-E), 4% just above the cliff, and a higher p99 TTFT (26.8 vs 18.6 s on 2 TiB); with NVMe well above W the codec can stay off. Its fixed cost is 1.0 GiB of GPU memory per GPU. FP8 KV, if acceptable, holds many of the same sessions and composes with the codec (§VIII-C).

Scope. We evaluate one engine per node with TP 1 and 2 (over PCIe); pipeline and expert parallelism keep the per-chunk record format but are unmeasured, as are several engines sharing one cache server. FP8 attention records average 0.84 of raw, but a page is sized for the worst layer (0.906) and the fused FP8 kernel takes 1.25–1.28× FA3 FP8's time, so FP8 KV pairs with tier mode (Table IV). Attention records are laid out for GQA; for multi-head latent attention the tier code applies to its latents (unmeasured) but the fused kernel does not. FP4 KV leaves a 2-bit exponent, too little to code, and in hybrid models the code covers attention layers only.

XI. Related Work

Tiered KV caches. LMCache [1], Mooncake [2], CachedAttention [3], SGLang HiCache [50], Strata [11], HCache [13] and Tutti [12] store raw KV or states; [14], [8], [15], [7], [18] characterize offloading or report deployments, and [69] simulates placement policies across GPU, DRAM and SSD. Replacement policies (LRU-K [70], 2Q [71], ARC [75]), exclusive multi-level caching [76] and miss-ratio curves (Mattson's stack distances [77], SHARDS [78], Belady's bound [79]) pick which chunks a tier keeps; the codec sets how many fit (§X). **Lossy KV compression.** CacheGen, KVTC, KIVI, KVQuant [23], QServe [24] and FP8 KV change outputs. EVICPRESS [39] chooses per context between lossy compression and eviction; Minima-KV [6] mixes lossy FP8 and 3-bit pages in one paged pool; D-Quant [34] entropy-codes quantized KV into fixed-size bitstreams decoded inside attention; KVComp [67] and PackKV [68] entropy-code quantized KV with fused kernels. AdaptCache [65] picks a lossy codec per entry and UltraQuant [66] serves 4-bit KV to agents. **Lossless compression.** SplitZip [5] targets prefill-to-decode transfer, LEXI [36] inter-chiplet links and TRACE [37] CXL hardware; DFloat11 [31], ZipNN [32] and ZipServ [35] compress weights, Huff-LLM [63] with Huffman decode in hardware and LCT [64] with GPU Huffman for training tensors ([17] also FP8 and KV tensors, offline); the concurrent ExANS [44] ANS-codes BF16 exponents for a KV store (Table II), but ANS state cannot be decoded inside attention; OpenZL [38], MPC [26] and ndzip-gpu [27] write contiguous outputs; Gorilla [40], Chimp [41], Elf [28] and BUFF [29] XOR or split floats, and ALP-RD [42] codes leading bits with a 3-bit dictionary plus exceptions, like our primary symbols; ExactKV adds a second escape level and, like FastLanes [43], a lane-parallel layout. **Attention over compressed KV.** Quantized-KV kernels dequantize inside attention and ZipServ decodes inside GEMM. The concurrent ZipFusion [33] decodes BF16 KV losslessly in FlashInfer's [25] decode attention (1.25× faster on an H100) from fixed 12-bit blocks, raw beyond 16 exponents (21% of our K blocks); it has no tiers. The concurrent GAAC [62] is closest to pool mode: calibrated per-channel base-plus-offset exponent codes (1.405× on BF16 KV, as ours), a fused attention kernel and fixed-capacity slots, on personal devices. ExactKV differs in four ways: it runs with unmodified vLLM and CUDA graphs; its records share pages with an overflow area (1.38× blocks against 1.33× for per-record slots on our KV, §VI); it codes FP8 KV; and the same blobs move through external tiers as exact-length objects. **Compressed buffer management.** The compression cache [51], compressed virtual memory [52] and zswap [53] keep a compressed level in front of a slower one; InnoDB [54] stores compressed pages in power-of-two blocks or punches holes (§V), and Umbra [55] and LeanStore [56] manage variable-size pages.

XII. Conclusion

ExactKV puts a lossless KV code into LMCache's tiers and vLLM's GPU pool without changing vLLM. A compressed tier acts as a raw tier 1.4× its bytes and a compressed pool as a raw pool of its token count; at LMCache's write-through cliff it serves 2.8–3.3× the goodput, bit-exactly.

Acknowledgment

Use of AI-generated content. Claude Code (Anthropic) and OpenAI Codex drafted text, figure, prototype and benchmark code, and ran experiments under the authors' direction. The authors checked every claim against logged runs and take full responsibility.

References

[1] Y. Liu, Y. Cheng, J. Yao et al., "LMCache: An efficient KV cache layer for enterprise-scale LLM inference," arXiv:2510.09665, 2025.
[2] R. Qin, Z. Li, W. He et al., "Mooncake: A KVCache-centric disaggregated architecture for LLM serving," FAST 2025.
[3] B. Gao, Z. He, P. Sharma et al., "Cost-efficient large language model serving for multi-turn conversations with CachedAttention," USENIX ATC 2024.
[4] Y. Liu et al., "CacheGen: KV cache compression and streaming for fast large language model serving," SIGCOMM 2024.
[5] Y. Guo and S. Joshi, "SplitZip: Ultra fast lossless KV compression for disaggregated LLM serving," arXiv:2605.01708, 2026.
[6] S. Kozyrev and D. Maiboroda, "Minima-KV: Retention-preserving KV cache compression with mixed-format paged attention," arXiv:2608.23834, 2026.
[7] Y. Zhang, X. Yang, H. Chen, F. Li et al., "PolarKV: Tier locally, serve globally—a KV cache over cloud memory and storage," PVLDB 19(12), 2026.
[8] W. Meng, B. Lee and H. Wang, "Understanding bottlenecks for efficiently serving LLM inference with KV offloading," arXiv:2601.19910, 2026.
[9] W. Kwon, Z. Li, S. Zhuang et al., "Efficient memory management for large language model serving with PagedAttention," SOSP 2023.
[10] L. Zheng, L. Yin, Z. Xie et al., "SGLang: Efficient execution of structured language model programs," NeurIPS 2024.
[11] Z. Xie, Z. Xu, M. Zhao, Y. An et al., "Strata: Hierarchical context caching for long context language model serving," arXiv:2508.18572, 2025.
[12] S. Qiu, Y. Hu, X. Wang, W. Zhu et al., "Tutti: Making SSD-backed KV cache practical for long-context LLM serving," arXiv:2605.03375, 2026.
[13] S. Gao, Y. Chen and J. Shu, "Fast state restoration in LLM serving with HCache," EuroSys 2025.
[14] J. Kanichai, T. De Matteis and A. Trivedi, "Building py-kvcache: A performance characterization of external KV caching for vLLM with NVMe SSDs," arXiv:2609.11744, 2026.
[15] Y. He, D. Shi, Y. Dai et al., "Characterizing predictability-latency trade-offs of KV-cache SSD offloading in LMCache for LLM serving systems," arXiv:2610.21249, 2026.
[16] T. Singh, "The KV cache is the new memory wall," arXiv:2609.30854, 2026.
[17] A. Heilper and D. Singer, "Lossless compression of neural network components: Weights, checkpoints, and K/V caches in low-precision formats," arXiv:2508.19263, 2025.
[18] Y. Deng, Z. You, L. Xiang et al., "AlayaDB: The data foundation for efficient and effective long-context LLM inference," arXiv:2504.10326, 2025.
[19] J. Wang, J. Han, X. Wei et al., "KVCache cache in the wild: Characterizing and optimizing KVCache cache at a large cloud provider," USENIX ATC 2025.
[20] SemiAnalysis, "InferenceX AgentX: Methodology and datasets," https://inferencex.semianalysis.com/agentx, 2026.
[21] B. Liu, H. Qiu, Í. Goiri, R. Fonseca et al., "Agentic coding in the wild: Characterizing GitHub Copilot traces at production scale," arXiv:2608.00101, 2026.
[22] Z. Liu, J. Yuan, H. Jin et al., "KIVI: A tuning-free asymmetric 2bit quantization for KV cache," ICML 2024.
[23] C. Hooper, S. Kim, H. Mohammadzadeh, M. W. Mahoney, Y. S. Shao, K. Keutzer and A. Gholami, "KVQuant: Towards 10 million context length LLM inference with KV cache quantization," NeurIPS 2024.
[24] Y. Lin, H. Tang, S. Yang et al., "QServe: W4A8KV4 quantization and system co-design for efficient LLM serving," MLSys 2025.
[25] Z. Ye et al., "FlashInfer: Efficient and customizable attention engine for LLM inference serving," MLSys 2025.
[26] A. Yang, H. Mukka, F. Hesaaraki and M. Burtscher, "MPC: A massively parallel compression algorithm for scientific data," IEEE Cluster 2015.
[27] F. Knorr, P. Thoman and T. Fahringer, "ndzip-gpu: Efficient lossless compression of scientific floating-point data on GPUs," SC 2021.
[28] R. Li et al., "Elf: Erasing-based lossless floating-point compression," PVLDB 16(7), 2023.
[29] C. Liu, H. Jiang, J. Paparrizos and A. J. Elmore, "Decomposed bounded floats for fast compression and queries," PVLDB 14(11), 2021.
[30] NVIDIA, "nvCOMP: GPU-accelerated compression and decompression library," https://developer.nvidia.com/nvcomp.
[31] T. Zhang, M. Hariri, S. Zhong et al., "70% size, 100% accuracy: Lossless LLM compression for efficient GPU inference via dynamic-length float," NeurIPS 2025.
[32] M. Hershcovitch, A. Wood, L. Choshen et al., "ZipNN: Lossless compression for AI models," IEEE CLOUD 2025.
[33] Y. Chen, P. Wang and H. Xu, "ZipFusion: Fusing lossless KV decompression with attention for efficient LLM decoding," arXiv:2610.33719, 2026.
[34] Y. Su, H. Liu, G. Yu and J. Zhu, "D-Quant: Driftable entropy coding for KV cache quantization," arXiv:2609.19880, 2026.
[35] R. Fan, X. Yu, X. Pan et al., "ZipServ: Fast and memory-efficient LLM inference with hardware-aware lossless compression," ASPLOS 2026.
[36] M. Sun, A. Kanani, K. Shroff et al., "LEXI: Lossless exponent coding for efficient inter-chiplet communication in hybrid LLMs," arXiv:2603.15589, 2026.
[37] R. Xie, A. U. Haq, Y. Fang, L. Ma et al., "TRACE: Unlocking effective CXL bandwidth via lossless compression and precision scaling," arXiv:2509.03377, 2025.
[38] Y. Collet, N. Terrell, W. F. Handte et al., "OpenZL: Using graphs to compress smaller and faster," ICDE 2026, arXiv:2605.09928.
[39] S. Feng, Y. Liu, H. Li et al., "EVICPRESS: Joint KV-cache compression and eviction for efficient LLM serving," arXiv:2512.14946, 2025.
[40] T. Pelkonen, S. Franklin, J. Teller et al., "Gorilla: A fast, scalable, in-memory time series database," PVLDB 8(12), 2015.
[41] P. Liakos, K. Papakonstantinopoulou and Y. Kotidis, "Chimp: Efficient lossless floating point compression for time series databases," PVLDB 15(11), 2022.
[42] A. Afroozeh, L. X. Kuffo and P. Boncz, "ALP: Adaptive lossless floating-point compression," Proc. ACM Manag. Data (SIGMOD) 1(4), 2023.
[43] A. Afroozeh and P. Boncz, "The FastLanes compression layout: Decoding >100 billion integers per second with scalar code," PVLDB 16(9), 2023.
[44] OpenLake, "ExANS: Achieving 600 GB/s+ decode throughput at 1.51x lossless compression," https://www.theopenlake.com/blog/exans-lossless-gpu-compression-for-bf16-kv-cache, 2026.
[45] J. Shah, G. Bikshandi, Y. Zhang, V. Thakkar, P. Ramani and T. Dao, "FlashAttention-3: Fast and accurate attention with asynchrony and low-precision," NeurIPS 2024.
[46] R. Teknium, J. Quesnelle and C. Guang, "Hermes 3 technical report," arXiv:2408.11857, 2024.
[47] SemiAnalysis, "cc-traces-weka-062126: Claude Code agent session traces," https://huggingface.co/datasets/semianalysisai/cc-traces-weka-062126, 2026.
[48] Y. Li, H. Jiang, Q. Wu et al., "SCBench: A KV cache-centric analysis of long-context methods," ICLR 2025.
[49] W. Zhao, X. Ren, J. Hessel, C. Cardie, Y. Choi and Y. Deng, "WildChat: 1M ChatGPT interaction logs in the wild," ICLR 2024.
[50] SGLang Team, "SGLang HiCache: Fast hierarchical KV caching with your favorite storage backends," LMSYS blog, https://lmsys.org/blog/2025-09-10-sglang-hicache/, 2025.
[51] F. Douglis, "The compression cache: Using on-line compression to extend physical memory," USENIX Winter Conference, 1993.
[52] P. R. Wilson, S. F. Kaplan and Y. Smaragdakis, "The case for compressed caching in virtual memory systems," USENIX ATC 1999.
[53] Linux kernel documentation, "zswap," https://docs.kernel.org/admin-guide/mm/zswap.html.
[54] Oracle, "InnoDB table and page compression," MySQL 8.0 Reference Manual, https://dev.mysql.com/doc/refman/8.0/en/innodb-compression.html.
[55] T. Neumann and M. J. Freitag, "Umbra: A disk-based system with in-memory performance," CIDR 2020.
[56] V. Leis, M. Haubenschild, A. Kemper and T. Neumann, "LeanStore: In-memory data management beyond main memory," ICDE 2018.

[57] K. Staniszewski and A. Łańcucki, "KV cache transform coding for compact storage in LLM inference," ICLR 2026.
[58] C.-P. Hsieh, S. Sun, S. Kriman et al., "RULER: What's the real context size of your long-context language models?" COLM 2024.
[59] K. Cobbe, V. Kosaraju, M. Bavarian et al., "Training verifiers to solve math word problems," arXiv:2110.14168, 2021.
[60] M. Zukowski, S. Héman, N. Nes and P. Boncz, "Super-scalar RAM-CPU cache compression," ICDE 2006.
[61] Apache Parquet, "Encodings: BYTE_STREAM_SPLIT," https://parquet.apache.org/docs/file-format/data-pages/encodings/.
[62] Y. Guo, A. M. George and S. Joshi, "GAAC: Lossless KV compression for bandwidth-efficient LLM inference on personal devices," arXiv:2610.38210 (preprint), 2026.
[63] P. Yubeaton, T. Mahmoud, S. Naga et al., "Huff-LLM: End-to-end lossless compression for efficient LLM inference," arXiv:2502.00922 (preprint), 2025.
[64] M. Zhu, B. Sun, P. Liò et al., "LCT: Losslessly compressed tensors with GPU friendly execution for training," arXiv:2610.22179 (preprint), 2026.
[65] S. Feng, H. Li, K. Du et al., "AdaptCache: KV cache native storage hierarchy for low-delay and high-quality language model serving," arXiv:2509.00105 (preprint), 2025.
[66] I. Chakrabarti, D. Limpus, A. G. Rana et al., "UltraQuant: 4-bit KV caching for context-heavy agents," arXiv:2606.20474 (preprint), 2026.
[67] B. Jiang, T. Yang, Y. Liu et al., "KVComp: A high-performance, LLM-aware, lossy compression framework for KV cache," arXiv:2509.00579 (preprint), 2025.
[68] B. Jiang, T. Yang, Y. Liu et al., "PackKV: Reducing KV cache memory footprint through LLM-aware lossy compression," IPDPS 2026.
[69] S. D. Tumkur, J. Iyer, M. Simhadri et al., "Where should the KV cache live? Placement policies across GPU, CPU, and SSD for long-lived sessions," arXiv:2609.16215 (preprint), 2026.
[70] E. J. O'Neil, P. E. O'Neil and G. Weikum, "The LRU-K page replacement algorithm for database disk buffering," SIGMOD 1993.
[71] T. Johnson and D. Shasha, "2Q: A low overhead high performance buffer management replacement algorithm," VLDB 1994.
[72] J. Yang, Y. Zhang, Z. Qiu, Y. Yue, and R. Vinayak, "FIFO queues are all you need for cache eviction," SOSP 2023.
[73] A. Zandieh, M. Daliri, M. Hadian and V. Mirrokni, "TurboQuant: Online vector quantization with near-optimal distortion rate," arXiv:2504.19874, 2025.
[74] J. Kübler, E. Kurtić, L. Wilkinson et al., "The state of FP8 KV-cache and attention quantization in vLLM," vLLM Blog, https://vllm.ai/blog/2026-04-22-fp8-kvcache, 2026.
[75] N. Megiddo and D. S. Modha, "ARC: A self-tuning, low overhead replacement cache," FAST 2003.
[76] T. M. Wong and J. Wilkes, "My cache or yours? Making storage more exclusive," USENIX ATC 2002.
[77] R. L. Mattson, J. Gecsei, D. R. Slutz and I. L. Traiger, "Evaluation techniques for storage hierarchies," IBM Systems Journal 9(2), 1970.
[78] C. A. Waldspurger, N. Park, A. Garthwaite and I. Ahmad, "Efficient MRC construction with SHARDS," FAST 2015.
[79] L. A. Belady, "A study of replacement algorithms for a virtual-storage computer," IBM Systems Journal 5(2), 1966.